TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

WEEKLY REPORT 3

Khảo sát không gian thiết kế shared L2 Cache
đánh giá bằng gem5

Team lead: PhD. Lê Minh Sơn
Mentor: Cao Văn Nam

Họ và tên SV: Choi Won Seok
MSSV: 24200032

Table of content

1. Progress Summary 1
2. Implementation Improvement 1
2.1 Simulation Model Update 1
2.2 DSE Automation Script Update 2
2.3 Data Parser Script 3
3. Simulation Results 4
3.1 Analysis 4
3.2 Summary 5
4. Future Goals 6

Weekly Report

1. Progress Summary

This week, I refined both the DSE automation script and the data parser to improve their flexibility and robustness. The automation script now accepts command-line arguments for key paths that were previously hardcoded or defined as relative paths, including the gem5 build, system configuration file, output directory, and workload directory. This makes the script easier to reuse across different simulation setups.

I also updated the simulation model to use four RISC-V MinorCPU cores instead of one. Although this is not yet the team's final CVA6-based system, as its implementation is still being finalized, it provides a closer approximation of the intended multicore setup. The current design space exploration focuses on L2 cache capacity, while associativity remains fixed at the project's specified 16-way configuration.

The parser has also been improved to accept arguments specifying the simulation output directories to process and the destination of the generated results file. Additionally, it now handles statistics with zero values, which gem5 may omit from stats.txt. When a statistic is missing from one core but present in others, the parser identifies the missing value and records it as zero. It also handles division-by-zero cases when calculating derived metrics, preventing errors during data processing.

2. Implementation Improvement

2.1 Simulation Model Update

Last week, the simulation model used a single RISC-V MinorCPU core. This week, I updated the model to use four cores to better match the project's end goal. Although the current model does not yet use the team's CVA6 core, as its implementation is still being finalized, it provides a suitable intermediate setup for studying cache behavior under a multicore workload.

Each core has its own private L1 instruction and data caches. The instruction cache is configured with a capacity of 16 KiB and 4-way associativity, while the data cache has a capacity of 32 KiB and 8-way associativity, matching the intended CVA6 cache configuration. Unlike the previous experiments, which explored different L1 cache configurations, these parameters are now fixed so that the design space exploration can focus on the shared L2 cache.

The L2 cache is configured with a fixed associativity of 16 ways, as specified by the project baseline, while its capacity is swept across five configurations: 256 KiB, 512 KiB, 1 MiB, 2 MiB, and 4 MiB. The remaining simulation parameters are kept constant

across all runs: LRU replacement policy, non-inclusive cache configuration, 32 MSHRs, and a 10-cycle L2 latency. The private L1 caches connect to the shared L2 through gem5's coherent crossbar (L2Xbar), which connects to main memory through a single DDR4-2400 channel.

For this stage, I chose to focus on L2 capacity rather than simultaneously exploring replacement policies, prefetchers, and indexing functions. Simulating four cores takes longer than the previous single-core experiments, so establishing a baseline first allows me to identify how cache capacity affects performance across different workloads. The results can then help identify scenarios where additional design parameters may have a meaningful impact. For example, replacement policies are most relevant when competing cache blocks cause evictions that affect subsequent accesses, so understanding the workload's access patterns and miss behavior helps guide which configurations are worth exploring. This approach reduces unnecessary simulations and provides a clearer basis for future experiments.

As for the workload configuration, the simulations use four types of synthetic workloads: loop, temporal, stream, and graph. Each workload is intended to exercise different memory-access patterns and provide insight into how the shared L2 cache behaves under different access conditions.

For each workload type, I run three configurations:

1. Single active workload: One core runs the selected workload while the other three cores remain idle.
2. Four identical workloads: All four cores execute the same workload simultaneously.
3. Mixed workloads: Each core executes a different workload, allowing the cache to be evaluated under a combination of memory-access patterns.

This produces nine workload configurations in total: four single-active-core runs, four four-core runs using identical workloads, and one mixed-workload run. With five L2 cache capacities evaluated for each configuration, the experiment consists of 45 simulation runs.

The results will provide a baseline for comparing how L2 capacity affects cache behavior under different levels of core activity and workload diversity. In subsequent experiments, these findings will help determine which additional design parameters, such as replacement policy, prefetching, and indexing functions, are worth investigating.

2.2 DSE Automation Script Update

The previous version of the automation script had several limitations that made it less flexible and maintainable. This update addresses the main issues by improving how simulation paths, sweep parameters, and output directories are handled.

Previously, the paths to the system configuration file, workload directory, and simulation output directory were hardcoded into the script. These paths are now provided through command-line arguments, allowing the same script to be reused across different simulation setups without modifying the source code.

The parameter sweep mechanism has also been improved. Previously, adding or removing a parameter required manually modifying the nested loops used to generate every configuration combination. The updated implementation stores the parameters and their candidate values in a Python dictionary and uses itertools.product() to automatically generate all possible combinations. This simplifies the code and makes it easier to expand or reduce the design space.

To prevent existing simulation results from being accidentally overwritten, the script now checks whether the specified output directory already exists. If it does, the script stops and displays a warning to select a different directory. The naming convention for individual simulation runs has also been made more compact. In addition, each run stores its swept parameter values in a JSON file, allowing the parser to retrieve the configuration directly rather than extracting it from the directory name. This approach is more maintainable, particularly as the number of swept parameters increases or parameter values become longer.

Despite these improvements, some limitations remain. The sweep parameters and their candidate values are still defined directly within the script, meaning that changes to the design space require modifying the source code. Switching between different system models may also require manually adjusting the parameter definitions to match each model's supported configuration options. Future updates will aim to externalize the sweep configuration and improve compatibility with different system models, further reducing the need for manual code changes.

2.3 Data Parser Script

The improvements to the data parser are relatively minor compared to those made to the automation script. The main change is the addition of command-line arguments to specify the simulation output directory containing the run statistics and the destination of the generated CSV file. This makes the parser more flexible and easier to reuse across different experiments.

The parser has also been updated to work with the revised sweeper. Instead of extracting configuration parameters from each run's directory name, it now reads them from the JSON file generated by the sweeper. This makes the data pipeline less dependent on the directory naming convention and easier to maintain as the number of swept parameters increases.

Additionally, the parser now handles statistics with zero values, which gem5 may omit from stats.txt. When a statistic is absent from one core's output but present in others, the parser records the missing value as zero. Safe division has also been implemented when

calculating derived metrics, preventing division-by-zero errors from interrupting the data-processing process.

3. Simulation Results

3.1 Analysis

After completing the 45 simulation runs, I collected the L2 miss rates from the parsed CSV file to examine how cache capacity affects different workload types. The results are summarized below.

- For one workload running on one core with the other three idle:

L2 Size	256K	512K	1M	2M	4M
loop	100.0%	100.0%	0.4%	0.4%	0.4%
temporal	71.9%	44.4%	31.2%	29.1%	25.0%
graph	88.3%	75.5%	51.9%	10.8%	10.7%
stream	100.0%	99.9%	99.9%	99.0%	76.5%

- Loop: The 768 KiB dataset exceeds the 256 KiB and 512 KiB cache capacities, resulting in 100% miss rates. At 1 MiB, the dataset fits in the cache, reducing misses to 0.4%.
- Temporal: The workload contains a 512 KiB hot region and a 4 MiB cold region. Increasing cache capacity allows more data to remain cached, reducing the miss rate from 71.9% to 25.0%.
- Graph: The workload randomly accesses a 2 MiB dataset. The miss rate decreases as cache capacity approaches the dataset size, dropping to around 10.8% at 2 MiB.
- Stream: The workload operates on three 2 MiB arrays, exceeding every tested cache size. Miss rates remain high, although the drop to 76.5% at 4 MiB is due to the hits in initialization phase.

Overall, the results show that the impact of L2 capacity depends on the workload's access pattern and working-set size.

- Result of all four cores running the same workload. Since each core runs a separate program with its own copy of the dataset, the total working set is approximately four times larger. Therefore, running four copies with an L2 of size (S) can be compared with running one copy with an L2 of size (S/4):

Workload	x4 at 1MiB vs Alone at 256KiB	x4 at 2MiB vs Alone at 512KiB	x4 at 4MiB vs Alone at 1MiB
loop	100% vs 100%	100% vs 100%	0.3% vs 0.4%
temporal	71.5% vs 71.9%	43.8% vs 44.4%	30.5% vs 31.2%

graph	88.2% vs 88.3%	75.2% vs 75.5%	51.3% vs 51.9%

The miss rates are nearly identical across the compared configurations. Per-core MPKI values inside the CSV also differ by less than 1%, suggesting that each core experiences similar cache behavior when running identical workloads. These results support the approximation that four copies sharing an L2 of size (S) behave similarly to one copy using an L2 of size (S/4). For the loop workload, this means a 4 MiB shared L2 is needed to reach the low-miss-rate region observed with a 1 MiB L2 on a single core.

- Lastly the result of running all four cores on different workload type, I'll be using the ratios of MPKI in for each workload to compare how inteference affect the miss rates:

L2 size	256K	512K	1M	2M	4M
temporal (core 0)	1.4x	2.1x	2.7x	2.0x	1.5x
stream (core 1)	1.0x	1.0x	1.0x	1.0x	1.3x
loop (core 2)	1.0x	1.0x	57x	3.4x	1.0x
graph (core 3)	1.1x	1.3x	1.8x	7.6x	5.0x

Interference is strongest when a workload's dataset is close to fitting in the L2 cache. At 1 MiB, the loop workload's MPKI increases from 0.70 to 39.4, while graph's MPKI rises significantly at 2 MiB. This suggests that competing workloads evict data that would otherwise remain cached.

Stream is largely unaffected at smaller cache sizes because it already has a high miss rate. However, at 4 MiB, interference reduces its limited data reuse at the initialization phases. Its continuous stream of new cache lines also contribute to evictions in other workloads.

Average L2 miss latency also increases under contention. For example, graph's average miss latency rises from 74.9–90.7 cycles when running alone to 93.0–99.0 cycles in the mixed workload. With a single DRAM channel, concurrent misses compete for memory bandwidth and increase queuing delays. Therefore, sharing the L2 can increase both the number of misses and their average latency.

3.2 Summary

To summarize, increasing L2 cache capacity generally reduces miss rates, but the improvement depends on each workload's working-set size and memory access pattern. When multiple cores share the cache, interference can reduce these benefits by causing additional evictions and increasing memory access latency. Therefore, the most suitable cache size depends not only on individual workloads but also on the combination of workloads running simultaneously.

Note: Since the full results.csv contains a large amount of data, this report presents only selected metrics for comparison. The complete results and additional metrics are available in the GitHub repository.

4. Future Goals

In the upcoming weeks, I plan to integrate the team's finalized CVA6 core into the simulation and continue exploring the L2 design space by testing replacement policies, prefetchers, and indexing functions. The goal is to identify which changes provide meaningful performance improvements. I will also study TCM in more depth to evaluate its feasibility and potential performance benefits.